

Sophix V3 DD Reading And Implementation Order

Date: 2026-06-01

Audience: dev leads, backend squads, frontend squads, QA, DevOps, product owners

Purpose: explain how to read and implement the SOPHIX V3 DD set without getting lost in the number of documents.

1. Mandatory First Read

Every squad starts with these root/control documents before reading a module DD:

Order	Document	Why read it
1	APPFREESN-HLD_SOPHIX_V3-DD_ALIGNED-20260601	Understand the full architecture, module boundaries, and deployment model.
2	APPFREESN-PROCESS_MODULE_MAP-DD_ALIGNED-20260601	Understand process ownership and module participation.
3	DD_INDEX	Fast navigation and canonical decisions.
4	DD_CROSS-00-System_Traceability_and_Implementation_Map	Feature-to-frontend-to-module traceability.
5	Sophix_V3_Performance_Complexity_Assessment_20260601	Understand what must be simplified and phased.
6	Sophix_V3_MVP_Runtime_Simplification_Baseline_20260601	Binding first-release implementation baseline.
7	DD_API-00-API_Catalog_and_Gateway_Standards	API conventions, idempotency, gateway rules, error model.
8	DD_QA-01-End_to_End_Test_Strategy	Required E2E and contract testing direction.

Rule: if a module DD appears to conflict with the simplification baseline, follow the baseline unless architecture governance approves an exception.

2. How To Read A Module

For each capability, read in this order:

1. Owning DD.
2. Its DEV_IMPL_GUIDE.
3. Upstream dependencies listed in the DD.
4. Downstream consumers/events listed in the DD.
5. Related frontend DD, if a UI uses it.
6. Relevant E2E scenarios.

Do not start implementation from a sample payload alone. Samples explain behavior, but ownership, idempotency, events, and state transitions are defined across the DD and guide.

3. Implementation Dependency Order

Wave 0 - Platform And Controls

Build or prepare:

- 01_foundations: auth, DB, Kafka, cache, Camunda, Drools, file storage.
- DD_API-00: API/gateway standards.
- DD_CROSS-00: traceability rules.
- DD_QA-01: test strategy.
- EM-CFG-03: RBAC catalog.
- EM-CFG-04: approval catalog.

Why: all later squads depend on identity, API conventions, events, workflow/rules, file access, permissions, and testing.

Wave 1 - Revenue-Critical Core

Build in this order:

Order	Capability	Key DDs
1	Customer/account/KYC basics	ILM-CFG-01, FOUNDATION_FILE_STORAGE
2	Catalog and serviceability basics	PLM-CFG-01, SIP-01, RLM-CFG-01, ILM-CFG-02, tax/wallet/equipment config as needed
3	Subscription master	SUB-LM-01
4	Subscription workflow framework	SUB-WF-FRAMEWORK-01, SUB-WF-FRAMEWORK-01-WORKERS
5	Billing/payment basics	BIL-01, BIL-01-PAY-01, BIL-02, BIL-02-READ-01, BIL-05 where prepaid/wallet is needed
6	First payment provider	PAY-GW-01 for the selected Kenya provider class
7	Work order and workforce basics	WO-01, WO-01-FRAMEWORK, WO-01-FLOW-INSTALLATION, canonical EM-02 Contractor and Staff Registry
8	Stock/equipment basics	OSR-01, OSR-INSTANCE-01, PLM-CFG-06
9	Fulfillment happy path	FUL-02 parent/framework and WIK happy path steps
10	Service activation	SUB-WF-ACTIVATE-01, FUL-03
11	Notifications/internal tasks	NOT-01, ICN-01
12	Basic ticketing	TCK-01 with basic support categories
13	Essential frontends	FE-APP-00, FE-APP-01, FE-APP-02, FE-APP-03
14	Minimal reporting	REP-01, FE-APP-05 core dashboards

Wave 1 success: a customer can be sold, captured, paid, installed, activated, invoiced/read, supported, and reported.

Wave 2 - Operational Hardening

Add:

- pause/resume/terminate;
- dunning and non-payment suspension;
- WO support and shifting;
- OSR-RMA and equipment swap;
- provisioning reconciliation;
- customer self-care;
- reporting exports/reconciliation;
- broader billing recovery/tax operations if required for launch.

Wave 3 - Advanced Commercial, Assurance, And Audit

Add:

- upgrade/downgrade/relocation/migration;
- FUL-09 and FUL-10 where migration/swap requires them;
- campaigns, discounts, CVM;
- ASR specialized flows on top of TCK;
- procurement and inventory audit;
- field audits as one field-audit capability;

- USSD limited self-care;
- offline mediation/rating;
- advanced franchise/sales attribution.

4. Frontend Reading Order

Frontend squads read:

1. FE-APP-00 and its guide.
2. DD_API-00.
3. EM-CFG-03.
4. The target app DD and guide.
5. DD_CROSS-00 feature rows for the screens they build.
6. Owning backend module DDs for every command and panel.

Frontend rule: every command has one owning module API. Every panel lists its read owner. Dense screens lazy-load panels and handle partial failure.

5. Backend Reading Order

Backend squads read:

1. DD_INDEX.
2. DD_CROSS-00.
3. DD_API-00.
4. Owning module DD.
5. Owning module DEV_IMPL_GUIDE.
6. Related foundation docs: DB, Kafka, cache, Camunda, Drools, file storage.
7. Upstream/downstream DDs named in cross-DD coordination.
8. DD_QA-01 tests.

Backend rule: one module owns each write. Other modules call APIs or consume events; they do not write another module's tables.

6. QA Reading Order

QA reads:

1. HLD.
2. Process map.
3. DD_QA-01.
4. DD_CROSS-00.
5. Owning DD and guide for the feature under test.
6. Frontend DD if the test includes UI.

QA should build tests in this order:

- API contract tests;
- module integration tests;
- event replay/idempotency tests;
- E2E journey tests;
- frontend smoke tests by role.

7. DevOps Reading Order

DevOps reads:

1. HLD deployment topology.
2. FOUNDATION_DB.
3. FOUNDATION_KAFKA.
4. FOUNDATION_CACHE.
5. FOUNDATION_CAMUNDA.
6. FOUNDATION_DRTOOLS.
7. FOUNDATION_AUTH.
8. MVP runtime simplification baseline.

DevOps rule: deploy capability groups first, not one service per DD. Split later only for throughput, isolation, scaling, release independence, or team ownership.

8. Canonical Warnings

- DD_EM-02-Contractor_and_Staff_Registry is canonical. The older Team/Agent EM-02 is superseded.
- ASR DDs are TCK ticket type/workflow satellites, not separate ticketing engines.
- FA-01/02/03 are one field-audit capability with audit_type.
- SUB-WF migration owns business operation; FUL-10 owns technical cross-tech fulfillment.
- REP-01 is mandatory for reporting dashboards and exports.
- MED/RAT are offline/batch in this design.
- USSD is a channel adapter, not a business backend.